8-1 关系型数据库介绍

提前说明

- 数据库操作是WEB开发的一个非常重要的环节，所以想要学WEB开发的同学，比如最流行的MySQL数据库。
- 但MySQL的教学不在本教程内，因此推荐大家自行学习，参考教程：https://www.51zxw.net/List.aspx?cid=1032

什么是数据库

- 开发中，我们离不开数据存储，简单的可以把数据存在文件中，比如 a.txt等。
- 但是问题也随之而来，简单的增加数据还好，但对于想要找到特定位置的数据并修改更新之，就变得比较复杂；然后再加上多线程，并发等问题，我们直接使用文件存数据的方式操作上就变得异常困难。
- 后来，出现了数据库管理系统（DBMS, DataBase Management System），它可以帮助我们快捷的操作文件夹/文件中的数据。
- 想要使用数据库管理系统，我们只需要学习它的使用语句即可，再也不用关心底层数据的增删查改。

常见的数据库管理系统

场景的数据库一般表现为软件，比如

- MySQL、Oracle、SQL Server、PostgreSQL、DB2、SQLite等等（关系型数据库）。
- Redis、Memcached、MangoDb等等（键值型数据库）。

关系型数据库

- 关系型数据库，是指采用了关系模型来组织数据的数据库，其以行和列的形式存储数据，可以类比EXCEL表格。
- 一张表可以包含多列，每一列都有一个标题字段。然后可以有N行，一行数据有多个字段。
- 一个数据库中可以包含多张表。

键值型数据库

- 按照键-值的形式存在数据库中。

8-2 使用pymysql之查询基础

需求场景

在命令行中通过SQL语句，我可以实现数据的增删查改，但是如何通过Python代码来操作数据库呢？

解决方式

- 本质上，MySQL是一个客户端-服务端架构的软件，我们就可以使用不同的客户端去连接服务端。
- 在命令中使用自带的客户端连接： `mysql -uroot -p12345`
- 使用专门数据库可视化软件连接：比如 navicate、workbench等软件
- **只要遵循MySQL的客户端-服务端之间通信的协议，任何编程语言都可以充当MySQL的客户端。**
- Python语言中，有一个第三方包，它就实现了MySQL的通信协议，我们可以使用它来操作数据库。

```
pip3 install PyMySQL
```

示例1：pymysql的基本使用

- 在数据库 `db` 中新建下面这张 `users`表

```
-- 新建一个数据库，名字是 db
create database db charset utf8;

-- 使用 db
use db;

-- 在db中新建一张 users表
CREATE TABLE `users` (
    `id` int(11) NOT NULL AUTO_INCREMENT,
    `name` varchar(255) COLLATE utf8_bin NOT NULL,
    `password` varchar(255) COLLATE utf8_bin NOT NULL,
    PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_bin AUTO_INCREMENT=1 ;


-- 插入两条数据
insert into users(name, password) values("liuxu","12345");
insert into users(name, password) values("liuxu2","12345");
```

- 通过Python代码连接数据库并查询数据

```
import pymysql

# 获取连接
conn = pymysql.connect(
    host='127.0.0.1',
    port=3306,
    user='root',
    password='12345',
    database='db',
    charset='utf8'
```

```
)

# 获取游标
cursor = conn.cursor()


# 准备SQL语句
sql = 'select * from users;'


print(cursor.execute(sql))      # 返回受影响的函数

print(cursor.fetchone())        # 取出一行数据,    (1, 'liuxu', '12345')
print(cursor.fetchmany(2))      # 取出N行数据
print(cursor.fetchall())        # 取出剩下所有数据
```

- 补充1：直接使用`cursor = conn.cursor()`得到的游标获取的数据按照元组的形式组织数据，仅有数据值没有字段名

```
# 此时指定为字典的形式，即可以看得见字段名和数据
...
cursor = conn.cursor(pymysql.cursors.DictCursor)

sql = 'select * from users;'
cursor.execute(sql)
print(cursor.fetchone())        # {'id': 1, 'name': 'liuxu', 'password':
'12345'}
```

8-3 使用pymysql之按条件查询

需求场景

很多时候，需要按照一定的要求查询数据，比如按照姓名查找特定用户的数据，或者查找id=2的用户数据，此时该怎么办？

解决方式

按照条件需求查找数据，这是MySQL基本的功能。使用pymysql时，我们使用特定的语法就可以实现。

示例1：找到id=2的用户数据

```
...


sql = 'select * from users where id = %s;'      # 使用 %s占位

cursor.execute(sql, 2)                        # execute()中第一个是sql语句，第二个是替换占位的条件

for d in cursor.fetchall():
    print(d)
```

示例2：查询有多个条件

```
...


sql = 'select * from users where id >= %s and id <= %s;'        # 使用 %s占位

cursor.execute(sql, [1, 2])         # 以为有两个参数，使用列表会元组的形式存放替换占位的条件

for d in cursor.fetchall():
    print(d)
```

示例3：按照字典的形式传参

```
...
cursor = conn.cursor(pymysql.cursors.DictCursor)

sql = 'select * from users where id >= %(start)s and id <= %(end)s;'

cursor.execute(sql, {"start": 1, "end": 2})
for d in cursor.fetchall():
    print(d)
```

8-4 使用pymysql之新增数据

示例1：增加数据，手动提交

```
...
cursor = conn.cursor(pymysql.cursors.DictCursor)


sql = 'insert into users(name, password) values("liuxu3", "123456");'
cursor.execute(sql)
conn.commit()           # 需要确认一次，否则数据库中不会保存记录
```

示例2：自动提交保存

```
import pymysql

# 获取连接
conn = pymysql.connect(
    host='127.0.0.1',
    port=3306,
    user='root',
    password='12345',
    database='db',
    charset='utf8',
    autocommit=True         # autocommit=True，表示execute后自动提交保存，无需再手动commit()
)

cursor = conn.cursor(pymysql.cursors.DictCursor)


sql = 'insert into users(name, password) values("liuxu4", "123456")'
cursor.execute(sql)
```

示例3：批量执行 executemany

```
import pymysql

# 获取连接
conn = pymysql.connect(
    host='127.0.0.1',
    port=3306,
    user='root',
    password='12345',
    database='db',
    charset='utf8',
    autocommit=True
)

cursor = conn.cursor(pymysql.cursors.DictCursor)


sql = 'insert into users(name, password) values(%s, %s)'    # 占位，注意和 "%s"的区别！！！
item_list = [("liuxu10", "12345"), ("liuixu11", "12345"), ("liuxu12", "12345")]
# 多个替换数据
```

```
cursor.executemany(sql, args=item_list)     # 返回受影响的函数
```

8-5 使用pymysql之修改和删除数据

修改和删除比较简单，分两步：

- 第一步：编写SQL语句 sql
- 第二步：使用 cursor.execute(sql)

示例

```
import pymysql

# 获取连接
conn = pymysql.connect(
    ...
    autocommit=True
)

cursor = conn.cursor(pymysql.cursors.DictCursor)

# 更新id=1的用户 name="LIUXU"
update_sql = 'update users set name="LIUXU" where id=1;'
cursor.execute(sql)

# 删除id>5的用户数据
delete_sql = 'delete from users where id>5;'
cursor.execute(sql)
```

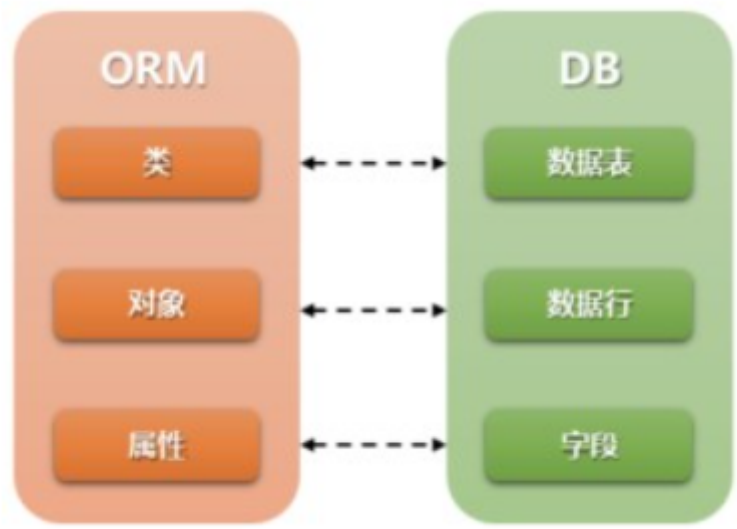
8-6 ORM介绍

需求场景

上面我们使用pymysql可以直接操作MySQL，但是你会发现**对数据的增删改查，都需要我们自己手写SQL语句。并且，存在大量重复的代码，不能使用面向对象**。那是否是否一种更优雅的方式来操作数据库呢？

解决方式

- 使用ORM来完成对数据的操作
- ORM：对象关系映射（Object-Relational Mapping），它可以将Python中的类-对象-属性 和数据库中的表-行数据-字段做映射。利用它，我们就不需要在直接操作数据库中的表、行和字段了，直接操作Python中面向对象的类、对象和属性即可实现数据的CRUD



- 优点：能够让一个不用sql语句的小白也能够通过python 面向对象的代码简单快捷的操作数据库
- 缺点：封装程度太高，有时候sql语句的效率相对较低，可能需要你自己写SQL语句

Python中的ORM典型代表

- Django-ORM，是Django框架自带的ORM，只能在django中使用
- SQLAlchemy ORM，是一个开源的ORM，不依赖web框架
- Peewee ，不依赖web框架的ORM
- 等等。

8-7 使用SQLAlchemy查询数据

安装：

- `pip3 install SQLAlchemy`

使用SQLAlchemy的基本流程分为三步：

- 创建session对象

- 创建类模型
- 使用session对象和模型类实现数据的CRUD

第一步：创建session对象

```
import pymysql
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker


# 使用pymysql作为MySQLdb
pymysql.install_as_MySQLdb()

# 指定连接的MySQL数据库
DATABASE_URL = "mysql://root:12345@localhost:3306/db"

# 创建引擎
engine = create_engine(DATABASE_URL)

# 基于引擎创建session
SessionLocal = sessionmaker(bind=engine)

# 实例化session对象，得到db对象
db = SessionLocal()

#  db使用只有需要关闭，避免占用资源
db.close()
```

第二步：创建模型类

```
from sqlalchemy import Column, String, Integer
from sqlalchemy.ext.declarative import declarative_base

# 创建对象的基类:
BaseModel = declarative_base()

# 定义User对象:
class User(BaseModel):
    __tablename__ = "users"     # 指定数据库中表的名字

    id = Column(Integer, primary_key=True, autoincrement=True)  # 定义id作为主键且
自增
    name = Column(String(255))          # 定义name字段，是一个string类型
    password = Column(String(255))      # password，是一个string类型

    # 人性化定制对象的打印输出
    def __str__(self):
        return f"id: {self.id}, name: {self.name}, password: {self.password}"
```

第三步：使用session对象和模型类实现数据的查询操作

- `result: User = db.query(User).filter(User.id == 1).first()`

- 必须使用 `first()`，否则只是SQL语句，不会执行查询操作。

```
import pymysql

from sqlalchemy import create_engine, Column, String, Integer
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker

pymysql.install_as_MySQLdb()

# 创建对象的基类:
BaseModel = declarative_base()


# 定义User对象:
class User(BaseModel):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    name = Column(String(255))
    password = Column(String(255))

    def __str__(self):
        return f"id: {self.id}, name: {self.name}, password: {self.password}"


DATABASE_URL = "mysql://root:12345@localhost:3306/db"
engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(bind=engine)
db = SessionLocal()

# 使用db和User来查询数据，查询出来的数据用来初始化User需要的属性，
# 即result是User的对象，它的每个属性值就是找出来的User表中每个字段的值。
result: User = db.query(User).filter(User.id == 1).first()
print(result)
print(result.id, result.name, result.password)


# 关闭db连接(重要)
db.close()
```

8-8 使用SQLAlchemy查询补充

除了上节课我们使用的 `db.query(User).filter(User.id == 1).first()` 来过滤查询数据外，还有很多其他的查询方式可以使用。

示例1：返回符合条件的所有数据 `all()`

```
db = SessionLocal()
result: User = db.query(User).filter(User.id >=5).all()
print(result)
```

示例2：使用 `filter_by()` 简化筛选参数

```
db = SessionLocal()
result: User = db.query(User).filter_by(id=2).first()
print(result)
```

示例3：主键筛选使用 `get()`，直接返回对象的示例

```
db = SessionLocal()
result: User = db.query(User).get(2)    # get(2), 表示查询主键id=2的数据
print(result)
```

示例4：使用 `order_by` 排序

- 默认是升序排列
- `db.query(User).order_by(User.id.desc()).all()` 使用 `字段.desc()` 指定降序排列

```
db = SessionLocal()
result: User = db.query(User).order_by(User.id).all()
print([r.id for r in result])
```

示例5：分页逻辑。

- `offset(n)` 指定过滤几行数据再开始查询
- `limit(n)` 指定每次查询几条数据

```
db = SessionLocal()
result: User = db.query(User).offset(0).limit(3).all()
print([r.id for r in result])
```

8-9 使用SQLAlchemy新增数据

示例1：新增数据

- 使用 `add(模型类对象)` 新增数据
- 必须使用 `db.commit()` 提交数据，否则新增的数据不会写到数据库中

```
db = SessionLocal()

db.add(User(name="liuxu", password="123456"))
db.add(User(name="liuxu2", password="123456"))
db.commit()     # 非常重要
```

示例2：批量新增数据，使用 `add_all()`

```
db = SessionLocal()
user3 = User(name="liuxu3", password="123456")
user4 = User(name="liuxu4", password="123456")
db.add_all([user3, user4])

db.commit()
```

示例3：批量新增数据，使用 `bulk_save_objects()`

```
db = SessionLocal()
user5 = User(name="liuxu5", password="123456")
user6 = User(name="liuxu6", password="123456")
db.bulk_save_objects([user5, user6])
db.commit()
```

8-10 使用SQLAlchemy修改和删除数据

修改数据和查询数据的原则：先查到数据，再修改或删除

示例1：修改数据

```
db = SessionLocal()

user: User = db.query(User).filter(User.id == 1).first()
print(user)
user.name = "LIUXU"     # 直接修改对象的属性值
db.commit()             # 然后提交保存才能更新
```

示例2：修改数据，使用 `update()`，支持批量更新

```
db = SessionLocal()
db.query(User).filter(User.id == 1).update({"name": "LLL"})     # 更新指定字段的值
db.commit()     # 提交保存修改
```

示例3：删除数据，使用 `delete()`，支持批量删除

```
db = SessionLocal()
db.query(User).filter(User.id >= 3).delete()        # 支持批量删除
db.commit()     # 提交才能删除
```

8-11 FastAPI集成SQLAlchemy之查询和删除

本节课，我们把FastAPI的知识点和SQLAlchemy的ORM结合起来，通过API接口直接操作MySQL中的数据。

本节课要点：

- 使用ORM的查询和删除操作
- 使用依赖注入的方式获取db，在请求来的时候获取db连接，在请求结束的时候关闭db连接。

完成代码示例

```
import typing

from fastapi import FastAPI, HTTPException, Depends
import pymysql
from sqlalchemy import create_engine, Column, String, Integer
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker, Session

pymysql.install_as_MySQLdb()


DATABASE_URL = "mysql://root:12345@localhost:3306/db"
engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(bind=engine)


def get_db():
    db: Session = SessionLocal()
    try:
```

```
        yield db
    finally:
        db.close()


BaseModel = declarative_base()


class User(BaseModel):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    name = Column(String(255))
    password = Column(String(255))

    def __str__(self):
        return f"id: {self.id}, name: {self.name}, password: {self.password}"


app = FastAPI(title="FastAPI + SqlAlchemy")


@app.get("/users")
def get_users(page: int = 1, size: int = 3, db: Session = Depends(get_db)):
    users: typing.List[User] = db.query(User).all()[(page - 1) * size:page *
size]
    return [{"id": u.id, "name": u.name} for u in users]


@app.get("/user/{user_id}")
def get_user_by_id(user_id: int, db: Session = Depends(get_db)):
    user: User = db.query(User).filter(User.id == user_id).first()
    if not user:
        raise HTTPException(detail=f"Not found user with id: {user_id}",
status_code=404)
    return {"id": user.id, "name": user.name}


@app.delete("/user/{user_id}")
def delete_user_by_id(user_id: int, db: Session = Depends(get_db)):
    db.query(User).filter(User.id == user_id).delete()
    return {"code": 1, "msg": "success"}
```

8-12 FastAPI集成SQLAlchemy之新建和更新

本节课要点：

- 使用pydantic的BaseModel做Schema接收请求体数据并做校验。

- 使用使用SQLAlchemy的ORM实现新建和更新数据操作。
- db.refresh(db_user) # refresh之后，db_user才有数据
- orm_mode = True # 设置后 response_model=UserOut时，才能直接返回 db_user

完整代码示例

```
import pymysql
from fastapi import FastAPI, HTTPException, Depends
from pydantic import BaseModel as SchemaBaseModel

from sqlalchemy import create_engine, Column, String, Integer
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker, Session

pymysql.install_as_MySQLdb()


DATABASE_URL = "mysql://root:12345@localhost:3306/db"
engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(bind=engine)


def get_db():
    db: Session = SessionLocal()
    try:
        yield db
    finally:
        db.close()


BaseModel = declarative_base()


class User(BaseModel):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    name = Column(String(255))
    password = Column(String(255))

    def __str__(self):
        return f"id: {self.id}, name: {self.name}, password: {self.password}"


app = FastAPI(title="FastAPI + SqlAlchemy")


class UserBase(SchemaBaseModel):
    name: str


class UserIn(UserBase):
    password: str


class UserOut(UserBase):
    id: int
```

```
    class Config:
        orm_mode = True     # 设置后 response_model=UserOut时，才能直接返回 db_user


@app.post("/user", response_model=UserOut)
def create_user(user: UserIn, db: Session = Depends(get_db)):
    # db_user = User(name=user.name, password=user.password)
    db_user = User(**user.dict())       # 和上面的用法是等价的
    db.add(db_user)
    db.commit()
    db.refresh(db_user)     # refresh之后，db_user才有数据
    return db_user


@app.put("/user/{user_id}", response_model=UserOut)
def update_user_by_id(user_id: int, user: UserIn, db: Session =
Depends(get_db)):
    db_user = db.query(User).filter(User.id == user_id).first()
    if not db_user:
        raise HTTPException(detail=f"Not found user with id: {user_id}",
status_code=404)
    db_user.name = user.name
    db_user.password = user.password
    db.commit()
    db.refresh(db_user)
    return db_user
```

8-13 项目代码结构调整

目前，我们已经实现了FastAPI和SQLAlchemy的集成，但是你会发现，项目的所有代码都在一个Python文件中。

这样会造成代码结构不清楚，层次不清晰，代码可读性差，维护成本高。

因此，我们需要做代码拆分，按照功能拆分成不同的Python文件。

项目完整代码示例（一个文件版本）

```
import typing
import pymysql
from fastapi import FastAPI, HTTPException, Depends
from pydantic import BaseModel as SchemaBaseModel

from sqlalchemy import create_engine, Column, String, Integer
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker, Session
```

```python
pymysql.install_as_MySQLdb()


DATABASE_URL = "mysql://root:12345@localhost:3306/db"
engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(bind=engine)


def get_db():
    db: Session = SessionLocal()
    try:
        yield db
    finally:
        db.close()


BaseModel = declarative_base()


class User(BaseModel):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    name = Column(String(255))
    password = Column(String(255))

    def __str__(self):
        return f"id: {self.id}, name: {self.name}, password: {self.password}"


app = FastAPI(title="FastAPI + SqlAlchemy")


class UserBase(SchemaBaseModel):
    name: str


class UserIn(UserBase):
    password: str


class UserOut(UserBase):
    id: int

    class Config:
        orm_mode = True     # 设置后 response_model=UserOut时，才能直接返回 db_user


@app.get("/users")
def get_users(page: int = 1, size: int = 3, db: Session = Depends(get_db)):
    users: typing.List[User] = db.query(User).all()[(page - 1) * size:page *
size]
    return [{"id": u.id, "name": u.name} for u in users]


@app.get("/user/{user_id}")
def get_user_by_id(user_id: int, db: Session = Depends(get_db)):
    user: User = db.query(User).filter(User.id == user_id).first()
```

```
    if not user:
        raise HTTPException(detail=f"Not found user with id: {user_id}",
status_code=404)
    return {"id": user.id, "name": user.name}


@app.delete("/user/{user_id}")
def delete_user_by_id(user_id: int, db: Session = Depends(get_db)):
    db.query(User).filter(User.id == user_id).delete()
    return {"code": 1, "msg": "success"}


@app.post("/user", response_model=UserOut)
def create_user(user: UserIn, db: Session = Depends(get_db)):
    # db_user = User(name=user.name, password=user.password)
    db_user = User(**user.dict())       # 和上面的用法是等价的
    db.add(db_user)
    db.commit()
    db.refresh(db_user)     # refresh之后，db_user才有数据
    return db_user


@app.put("/user/{user_id}", response_model=UserOut)
def update_user_by_id(user_id: int, user: UserIn, db: Session =
Depends(get_db)):
    db_user = db.query(User).filter(User.id == user_id).first()
    if not db_user:
        raise HTTPException(detail=f"Not found user with id: {user_id}",
status_code=404)
    db_user.name = user.name
    db_user.password = user.password
    db.commit()
    db.refresh(db_user)
    return db_user
```

拆分后的项目结构

```
── sql_app
    ├── main.py             # 程序入口
    ├── crud.py             # 增删查改的方法
    ├── database.py         # 数据库相关
    ├── models.py           # ORM模型类相关
    └── schemas.py          # Pydantic的BaseModel，校验相关
```

- database.py

```
import pymysql

from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, Session

pymysql.install_as_MySQLdb()
```

```
DATABASE_URL = "mysql://root:12345@localhost:3306/db"
engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(bind=engine)


def get_db():
    db: Session = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

- models.py

```
from sqlalchemy import Column, String, Integer
from sqlalchemy.ext.declarative import declarative_base


BaseModel = declarative_base()


class User(BaseModel):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    name = Column(String(255))
    password = Column(String(255))

    def __str__(self):
        return f"id: {self.id}, name: {self.name}, password: {self.password}"
```

- schemas.py

```
from pydantic import BaseModel


class UserBase(BaseModel):
    name: str


class UserIn(UserBase):
    password: str


class UserOut(UserBase):
    id: int

    class Config:
        orm_mode = True
```

- crud.py

```
import typing

from fastapi import HTTPException
```

```
from sqlalchemy.orm import Session

from models import User
from schemas import UserIn


def get_users(page: int, size: int, db: Session) -> typing.List[User]:
    users: typing.List[User] = db.query(User).all()[(page - 1) * size:page *
size]
    return users


def get_user_by_id(user_id: int, db: Session) -> User:
    user: User = db.query(User).filter(User.id == user_id).first()
    if not user:
        raise HTTPException(detail=f"Not found user with id: {user_id}",
status_code=404)
    return user


def delete_user_by_id(user_id: int, db: Session):
    db.query(User).filter(User.id == user_id).delete()


def create_user(user: UserIn, db: Session) -> User:
    # db_user = User(name=user.name, password=user.password)
    db_user = User(**user.dict())  # 和上面的用法是等价的
    db.add(db_user)
    db.commit()
    db.refresh(db_user)  # refresh之后，db_user才有数据
    return db_user


def update_user(user_id: int, user: UserIn, db: Session) -> User:
    db_user = db.query(User).filter(User.id == user_id).first()
    if not db_user:
        raise HTTPException(detail=f"Not found user with id: {user_id}",
status_code=404)
    db_user.name = user.name
    db_user.password = user.password
    db.commit()
    db.refresh(db_user)
    return db_user
```

- main.py

```
import typing

from fastapi import FastAPI, Depends
from sqlalchemy.orm import Session

import crud
from models import User
from database import get_db
from schemas import UserIn, UserOut

app = FastAPI(title="FastAPI + SqlAlchemy")
```

```python
@app.get("/users", response_model=typing.List[UserOut])
def get_users(page: int = 1, size: int = 3, db: Session = Depends(get_db)):
    user = crud.get_users(page, size, db)
    return user


@app.get("/user/{user_id}", response_model=UserOut)
def get_user_by_id(user_id: int, db: Session = Depends(get_db)):
    user: User = crud.get_user_by_id(user_id, db)
    return user


@app.delete("/user/{user_id}")
def delete_user_by_id(user_id: int, db: Session = Depends(get_db)):
    crud.delete_user_by_id(user_id, db)
    return {"code": 1, "msg": "success"}


@app.post("/user", response_model=UserOut)
def create_user(user: UserIn, db: Session = Depends(get_db)):
    return crud.create_user(user, db)


@app.put("/user/{user_id}", response_model=UserOut)
def update_user_by_id(user_id: int, user: UserIn, db: Session =
Depends(get_db)):
    return crud.update_user(user_id, user, db)
```